COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S1
DATE PERFORMED:	09/08/26	**DATE SUBMITTED:**	09/10/26
NAME:	Pangilinan Jr., Bienvenido B.	**INSTRUCTOR:**	Engr. Jimlord Quehado

ACTIVITY NO. 6 **[ACTIVITY NAME] Searching Techniques**

1. PROCEDURE OUTPUT

What procedures did you perform?

Table 7-1. Array of Values for Sort Algorithm Testing

Source Code

```
#include <iostream>
#include <vector>
#include <cstdlib>
#include <ctime>
#include "SortingAlgo.h"

std::vector<int> generateRandomValues() {
    srand( Seed: time(0));
    std::vector<int> values( n: 100);
    for (int& num : values) num = rand() % 100 + 1;
    return values;
}
int main() {
    auto numbers :vector<int>  = generateRandomValues();
    for (int n : numbers) std::cout << n << " ";
    std::cout << std::endl;

    return 0;
}
```

Console Output

```
C:\Users\Bienvenido\CLionProjects\C++\HOA7.1\cmake-build-debug\HOA7_1.exe
65 95 92 19 20 49 32 48 66 41 76 73 67 28 28 26 26 15 99 83 57 64 100 22 16 32 18 63 45 26 28 93 30 74 47 36 79 8 89 87
62 78 54 27 60 31 65 33 49 58 53 21 47 82 57 59 39 79 25 75 13 93 62 71 30 3 91 28 38 83 45 1 55 91 4 14 22 23 71 33 20
6 77 61 5 43 73 92 80 73 32 27 40 16 73 19 65 6 13 58

Process finished with exit code 0
```

Observation
- When running the code, it generates and outputs a vector of 100 random integers between 1 to 100, since there is **srand(time(0))** the code automatically runs random value everytime you run the code and it is restricted within 100, it loop until it display all the 100 random integers.

Table 7-2. Bubble Sort Technique

Source Code

```
// 1. Bubble Sort Technique
std::copy( first: originalArr,  last: originalArr + maxSize,  result: workArr);
std::cout << "\nBubble Sort Technique:\n";
bubbleSort(workArr, maxSize);
displayArr(workArr, maxSize);
```

```
template <typename T>
void bubbleSort(T arr[], int arrsize) {
    for (int i = 0; i < arrsize - 1; i++) {
        for (int j = 0; j < arrsize - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                std::swap( a: [&] arr[j],  b: [&] arr[j + 1]);
            }
        }
    }
}
```

Console Output

```
Bubble Sort Technique:
2 4 7 9 9 11 11 11 11 11 13 14 17 19 19 20 20 22 27 28 30 31 31 33 34 34 34 36 38 39 40 40 40 41 42 43 45 45 45 46 46 46
 51 53 54 55 56 56 57 58 58 58 58 59 59 60 60 60 61 61 62 64 64 64 64 66 67 67 69 70 70 71 72 72 73 74 75 75 76 77 77 78
 79 80 81 81 82 85 86 86 87 89 92 92 93 93 96 96 97 100
```

Observation
- The time complexity is O(n^2) since it is a nested loop. The code copies the unsorted array and sorts them out using the bubble sort, in which it compares the adjacent element from left to right during each pass and then the numbers are ordered from the smallest to largest correctly.

Table 7-3. Selection Sort Algorithm

Source Code

```
// 2. Selection Sort Algorithm
std::copy( first: originalArr,  last: originalArr + maxSize,  resultworkArr);
std::cout << "\nSelection Sort Algorithm:\n";
selectionSort(workArr, maxSize);
displayArr(workArr, maxSize);
```

```
template <typename T>
void selectionSort(T arr[], int arrSize) {
    for (int i = 0; i < arrSize - 1; i++) {
        int pos = routine_smallest(arr, arrSize,  k: i);
        std::swap( a: [&] arr[i],  b: [&] arr[pos]);
    }
}
```

```cpp
template <typename T>
int routine_smallest(const T A[], int max_size, int k) {
    int position = k;
    T smallestElement = A[k];

    for (int j = k + 1; j < max_size; j++) {
        if (A[j] < smallestElement) {
            smallestElement = A[j];
            position = j;
        }
    }
    return position;
}
```

Console Output

```
Selection Sort Algorithm:
2 4 7 9 9 11 11 11 11 11 13 14 17 19 19 20 20 22 27 28 30 31 31 33 34 34 34 36 38 39 40 40 40 41 42 43 45 45 45 46 46 46
 51 53 54 55 56 56 57 58 58 58 58 59 59 60 60 60 61 61 62 64 64 64 64 66 67 67 69 70 70 71 72 72 73 74 75 75 76 77 77 78
 79 80 81 81 82 85 86 86 87 89 92 92 93 93 96 96 97 100
```

Observation

- The time complexity of this code is O(n^2) due to being nested loop, the code works by scanning the unsorted numbers and find the smallest one before swapping in the front, after that in every step, it searches the remaining unsorted part for the minimum number or value and move it its designated corrected spot on the left, it loops the process until all of the numbers are arranged in correct manner.

Table 7-4. Insertion Sort Algorithm

Source Code

```
// 3. Insertion Sort Algorithm
std::copy( first: originalArr, last: originalArr + maxSize, result: workArr);
std::cout << "\nInsertion Sort Algorithm:\n";
insertionSort(workArr, maxSize);
displayArr(workArr, maxSize);
```

```
template <typename T>
void insertionSort(T arr[], int arrsize) {
    int k = 1, j;
    T temp;
    while (k < arrsize) {
        temp = arr[k];
        j = k - 1;
        while (j >= 0 && temp < arr[j]) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = temp;
        k++;
    }
}
```

Console Output

```
Insertion Sort Algorithm:
2 4 7 9 9 11 11 11 11 11 13 14 17 19 19 20 20 22 27 28 30 31 31 33 34 34 34 36 38 39 40 40 40 41 42 43 45 45 45 46 46 46
 51 53 54 55 56 56 57 58 58 58 58 59 59 60 60 60 61 61 62 64 64 64 64 66 67 67 69 70 70 71 72 72 73 74 75 75 76 77 77 78
 79 80 81 81 82 85 86 86 87 89 92 92 93 93 96 96 97 100
```

Observation

- Its time complexity is O(n^2) and the code works by dividing it into a sorted part and unsorted part on right side. The code takes an element from the unsorted part then it checks its values on the left side and also it shifts larger number to make some space before placing the element in its designated location, this loops every item repeatedly inserting each unsorted element to its correct spot until all the elements are in correct order.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given data sorting problems using appropriate basic sorting algorithms

Candidate 1 Bo Dalton Capistrano
Candidate 2 Cornelius Raymon Agustín
Candidate 3 Deja Jayla Bañaga
Candidate 4 Lalla Brielle Yabut
Candidate 5 Franklin Relano Castro
List of Candidates

Problem: Generate an array A[0…100] of unsorted elements, wherein the values in the array are indicative of a vote to a candidate. This means that the values in your array must only range from 1 to 5. Using sorting and searching techniques, develop an algorithm that will count the votes and indicate the winning candidate.

NOTE: The sorting techniques you have the option of using in this activity can be either bubble, selection, or insertion sort. Justify why you chose to use this sorting algorithm.

Deliverables:
Pseudocode of Algorithm
Screenshot of Algorithm Code
Output Testing

Pseudocode of Algorithm
ALGORITHM insertionSort(arr, arrsize) k = 1 WHILE k is less than arrsize temp = element at arr[k] j = k - 1 WHILE j is 0 or greater AND temp is less than element at arr[j] shift element at arr[j] to position arr[j + 1] decrease j by 1 END WHILE place temp into position arr[j + 1] increase k by 1 END WHILE END ALGORITHM

Screenshot of Algorithm Code

```
template <typename T>
void insertionSort(T arr[], int arrsize) {
    int k = 1, j;
    T temp;
    while (k < arrsize) {
        temp = arr[k];
        j = k - 1;
        while (j >= 0 && temp < arr[j]) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = temp;
        k++;
    }
}
```

Output Testing

```
C:\Users\Bienvenido\CLionProjects\C++\HOA7.1\cmake-build-debug\HOA7_1.exe
UNSORTED ARRAY:
1 3 3 5 5 1 5 4 1 5 2 4 1 5 3 4 5 3 2 2 3 5 4 5 5 5 1 2 4 5 4 3 3 1 1 1 1 2 4 4 5 5 4 3 4 4 2 5 4 2 4 4 4 5 2 3 1 4 2 2
4 4 1 5 3 4 5 1 1 1 5 2 2 4 2 4 1 3 2 5 1 3 5 2 2 4 4 2 4 4 5 3 5 1 1 3 1 4 3 2

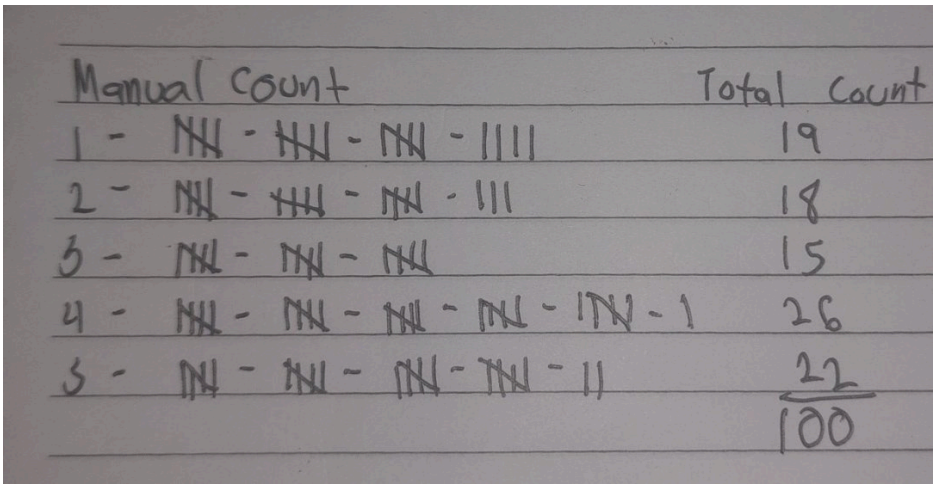
SORTED ARRAY:
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4
4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5

ELECTION RESULTS:
Candidate 1 - Bo Dalton Capistrano: 19 votes
Candidate 2 - Cornelius Raymon Agustin: 18 votes
Candidate 3 - Deja Jayla Banaga: 15 votes
Candidate 4 - Lalla Brielle Yabut: 26 votes
Candidate 5 - Franklin Relano Castro: 22 votes

WINNER: Candidate 4 - Lalla Brielle Yabut with 26 votes!

Process finished with exit code 0
```

Manual Count



Question: Was your developed vote counting algorithm effective? Why or why not?

- I think the counting algorithm i made was effective since it managed to display all 100 votes and tallied all the votes. Sorting the array with the insertion sort algorithm allowed the votes to be systematically organized in ascending order before counting, making it straightforward to group identical candidate IDs together and verify that all 100 votes were accurately processed and clean.

Conclusion:

- In this laboratory activity, I learned the three sorting techniques which are the bubblesort, selection sort and insertion sort and I saw these three have the same time complexity of O(n^2) due to them being nested loops. First is bubble sort, repeatedly compares neighboring elements and swaps them if they

are in the wrong order. As for the selection sort, it works by finding the smallest element in the unsorted part of the array and swapping it into its correct position. Finally, insertion sort builds the sorted list one item at a time by picking the next element and shifting larger numbers out of the way to put it in the right spot.

As for the supplementary part, I used the codes from the previous code to generate an array of 100 random numbers ranging from 1 to 5 for voting. Also I use the insertion sort here since it was the most effective one to avoid troubles when swapping. Once the array was sorted, the total counts for candidates also done, it will enter a final check identifying the candidate with the highest number of votes as the winner.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.